User and Group Administration

1. What is a user ?
- it is a security principal which is used to access any resource of a system.
- Resources means documents,photos,videos,storage,hardware like printer.

2. What is user Account ?
- It is a object which creates a security boundary for a user.

3. What is credential ?
- It is a combination of username and authentication mechanism/method.

4. What is Authentication ?
- It is a process by which a system checks the login credential is correct or not.

5. What are the authentication process ?
- username & password - Biometric (Fingerprint,IRIS, retina Scan) - Software Key
- Hardware token (Fido Key) - Authenticator App - Microsoft Passport - Windows Hello
- Picture Password - capcha - sound verification

Users in Linux
==========
1. Superuser : It has the full functions , full access to the system. UID is 0 . in linux superuser is known as root.

2. System User : it is dedicated for a particular job. as a person we can not use them. these are used by system processes and services. UID Range 1-999. Example : shutdown user, restart user , halt user.

3. Normal User : It is for a normal user. It has no administrative privilege. UID Range >= 1000. It can be promoted as administrator user.

Files responsible for user management

==========

1. /etc/passwd : it holds all information about a user apart from password.
2. /etc/shadow : It holds the password and the password and account policies for the user.

Login shell : it is the interface where user log in and executes commands or instructions. example : bash shell (bash : bourne Again shell) , k Shell , nologin shell , zsh shell , sh shell

Home Directory :

- it is location where user keeps all its data.
- when we create a user and user logs in for the first time system allocates a space for the user.
- If the user is root then home directory is /root
- if the user is a normal user then the home directory is /home/username
- Userdata includes Desktop,Documents,Photos,Videos,Links,Downloads etc.
- In linux we can customize the location of the user home directory during user creation.

Creating a user and setting password

```
[root@node1 ~]# useradd rohit
[root@node1 ~]# passwd rohit
Changing password for user rohit.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@node1 ~]#
```

User verification

```
[root@node1 ~]# id rohit
uid=1002(rohit) gid=1002(rohit) groups=1002(rohit)
[root@node1 ~]#
[root@node1 ~]# cat /etc/passwd | grep rohit
rohit:x:1002:1002::/home/rohit:/bin/bash
[root@node1 ~]#
[root@node1 ~]# grep rohit /etc/passwd
rohit:x:1002:1002::/home/rohit:/bin/bash
[root@node1 ~]#
```

User login

```
[root@node1 ~]# su - rohit
Last login: Tue Nov 19 00:16:48 EST 2024 on pts/0
[rohit@node1 ~]$
[rohit@node1 ~]$
[rohit@node1 ~]$ su - pallab
Password:
[pallab@node1 ~]$
```

Removing password from a user

```
[root@node1 ~]# passwd anas -d
Removing password for user anas.
passwd: Success
[root@node1 ~]#
[root@node1 ~]# logout
[student@node1 ~]$
[student@node1 ~]$ su - anas
[anas@node1 ~]$
```

Changing password from a user

```
[pallab@node1 ~]$ passwd
Changing password for user pallab.
Current password:
New password:
Retype new password:
passwd: all authentication tokens updated successfully.
[pallab@node1 ~]$
```

Or we can use standard input to set password to the user

```
[root@srv1 ~]# echo "redhat" | passwd --stdin raja
Changing password for user raja.
passwd: all authentication tokens updated successfully.
[root@srv1 ~]#
```

Note : when the user sets its password , the password must be 8 characters and complex.

Creating a user with custom user id

```
[root@node1 ~]# useradd -u 1008 jeet
[root@node1 ~]# passwd jeet
Changing password for user jeet.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@node1 ~]# id jeet
uid=1008(jeet) gid=1008(jeet) groups=1008(jeet)
[root@node1 ~]#
```

Changing user id of an existing user

```
[root@node1 ~]# usermod -u 1234 biraj
[root@node1 ~]#
[root@node1 ~]# id biraj
uid=1234(biraj) gid=1005(biraj) groups=1005(biraj)
[root@node1 ~]#
```

Creating a user with description and custom user id

```
[root@node1 ~]# useradd -u 1346 -c "Network Admin" arijit
```

```
[root@node1 ~]# grep arijit /etc/passwd
arijit:x:1346:1346:Network Admin:/home/arijit:/bin/bash
[root@node1 ~]#
```

Adding description for an existing user

```
[root@node1 ~]# grep biraj /etc/passwd
biraj:x:1234:1005::/home/biraj:/bin/bash
[root@node1 ~]#
[root@node1 ~]# usermod -c "Devops Engineer" biraj
[root@node1 ~]#
[root@node1 ~]# grep biraj /etc/passwd
biraj:x:1234:1005:Devops Engineer:/home/biraj:/bin/bash
[root@node1 ~]#
```

Creating a user with a non interactive login shell

```
[root@node1 ~]# useradd -s /sbin/nologin david
[root@node1 ~]#
[root@node1 ~]# grep david /etc/passwd
david:x:1347:1347::/home/david:/sbin/nologin
```

```
[root@node1 ~]# su - david
This account is currently not available.
```

Changing login shell of an existing user to a non-interactive login shell

```
[root@node1 ~]# usermod -s /sbin/nologin biraj
[root@node1 ~]#
[root@node1 ~]# su - biraj
Last login: Wed Nov 20 23:46:47 EST 2024 on pts/0
This account is currently not available.
```

Changing a non interactive user to interactive user

```
[root@node1 ~]# usermod -s /bin/bash biraj
[root@node1 ~]#
[root@node1 ~]# su - biraj
Last login: Wed Nov 20 23:47:11 EST 2024 on pts/0
[biraj@node1 ~]$
```

To lock a user

```
[root@node1 ~]# usermod -L biraj
[root@node1 ~]#
```

To unlock a locked user

```
[root@node1 ~]# usermod -U biraj
[root@node1 ~]#
```

Real time scenario-1

==========

We want to create a user named akash, but I dont have space in the home directory , but this akash user needs a large amount of space in the home directory.

Find solution ⇒

We can change the location of the home directory during user creation.

Creating custom home directory for a user

```
[root@node1 ~]# useradd -d /tmp/akashstore akash
[root@node1 ~]#
[root@node1 ~]# grep akash /etc/passwd
akash:x:1348:1348::/tmp/akashstore:/bin/bash
[root@node1 ~]#
```

Real time scenario-2

==========

Q. abc.com company wants some custom settings for their users during creation,

1. Home directory = /userdata
2. Default login shell = /sh

Solution : we need to change in useradd file inside /etc/default location

1. Open /etc/defaults/useradd file
2. Change the parameters as per required
3. Save and exit

To test create an user and check the changed details

Open the useradd file

```
[root@node1 ~]# vim /etc/default/useradd
```

```
# useradd defaults file
GROUP=100
HOME=/tmp/newusers
INACTIVE=-1
EXPIRE=
SHELL=/sh
SKEL=/etc/skel
CREATE_MAIL_SPOOL=yes

~
```

Exit and create a new user

```
[root@node1 ~]# useradd ishan
[root@node1 ~]# grep ishan /etc/passwd
ishan:x:1349:1349::/tmp/newusers/ishan:/sh
[root@node1 ~]#
```

To delete the user & its home directory

```
[root@node1 ~]# userdel -r david
[root@node1 ~]#
[root@node1 ~]# ls /home/
anas  arijit  biraj  devops  jeet  pallab  rohit  student
[root@node1 ~]#
```

Creating multiple user using a template

```
[root@node1 ~]# vim userlist
```

```
student:x:1000:1000::/home/student:/bin/bash
student1:x:2001:2001::/home/student1:/bin/bash
student2:x:2002:2002::/home/student2:/bin/bash
student3:x:2003:2003::/home/student3:/bin/bash
```

Save and exit

```
[root@node1 ~]# newusers userlist
[root@node1 ~]# grep student /etc/passwd
student:x:1000:1000::/home/student:/bin/bash
student1:x:2001:2001::/home/student1:/bin/bash
student2:x:2002:2002::/home/student2:/bin/bash
student3:x:2003:2003::/home/student3:/bin/bash
```

Manage User Passwords

Objectives

Set a password management policy for users, and manually lock and unlock user accounts.

Shadow Passwords and Password Policy

Originally, encrypted passwords were stored in the world-readable `/etc/passwd` file. This was considered adequate until dictionary attacks on encrypted passwords became common. The encrypted passwords were moved to the `/etc/shadow` file, which only the `root` user can read.

Like the `/etc/passwd` file, each user has an entry with in the `/etc/shadow` file. An example entry from the `/etc/shadow` file has nine colon-separated fields:

```
[root@host ~]# cat /etc/shadow
...output omitted...
user03:$6$CSsXsd3rwghsdfarf:17933:0:99999:7:2:18113:
```

Each field of this code block is separated by a colon:

- **`user03`** : Name of the user account.
- **`$6$CSsXsd3rwghsdfarf`** : The encrypted password of the user.
- **`17933`** : The days from the epoch when the password was last changed, where the epoch is `1970-01-01` in the UTC time zone.
- **`0`** : The minimum days since the last password change before the user can change it again.
- **`99999`** : The maximum days without a password change before the password expires. An empty field means that the password never expires.
- **`7`** : The number of days ahead to warn the user that their password will expire.
- **`2`** : The number of days without activity, starting with the day the password expired, before the account is automatically locked.
- **`18113`** : The day when the account expires in days since the epoch. An empty field means that the account never expires.
- The last field is typically empty and reserved for future use.

Format of an Encrypted Password

The encrypted password field stores three pieces of information: the hashing algorithm in use, the *salt*, and the encrypted hash. Each piece of information is delimited by the dollar ($) character.

```
$6$CSsXcYG1L/4ZfHr/$2W6evvJahUfzfHpc9X.45Jc6H30E
```

- **`6`** : The hashing algorithm in use for this password. A 6 indicates a SHA-512 hash, the RHEL 9 default, a 1 indicates MD5, and a 5 indicates SHA-256.
- **`CSsXcYG1L/4ZfHr/`** : The salt in use to encrypt the password; originally chosen at random.
- **`2W6evvJahUfzfHpc9X.45Jc6H30E`** : The encrypted hash of the user's password; combining the salt and the unencrypted password and then encrypting to generate the password hash.

Password Verification

When a user tries to log in, the system looks up the entry for the user in the `/etc/shadow` file, combines the salt for the user with the unencrypted typed password, and encrypts the combination of the salt and unencrypted password with the specified hashing algorithm. If the result matches the encrypted hash, then the user typed the right password. If the result does not match the encrypted hash, then the user typed the wrong password and the login attempt fails. This method allows the system to determine whether the user typed the correct password without storing that password in a usable form for logging in.

To create a primary group , we need to create a user

```
[root@ip-172-31-95-150 ~]# useradd mouli
[root@ip-172-31-95-150 ~]# passwd mouli
Changing password for user mouli.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@ip-172-31-95-150 ~]#
```

To create a secondary group

```
[root@ip-172-31-95-150 ~]# groupadd managers
```

To see the list of groups

```
[root@ip-172-31-95-150 ~]# tail -n 5 /etc/group
chrony:x:994:
sgx:x:993:
ec2-user:x:1000:
managers:x:1001:
mouli:x:1002:
[root@ip-172-31-95-150 ~]#
```

To add a user to a secondary group

```
[root@ip-172-31-95-150 ~]# usermod -aG managers mouli
[root@ip-172-31-95-150 ~]# grep managers /etc/group
managers:x:1001:mouli
[root@ip-172-31-95-150 ~]#
```

-a : append

-G : adding to a group

```
[root@ip-172-31-95-150 ~]# groupadd admins
[root@ip-172-31-95-150 ~]#
[root@ip-172-31-95-150 ~]# usermod -G admins mouli
[root@ip-172-31-95-150 ~]#
[root@ip-172-31-95-150 ~]# grep mouli /etc/group
mouli:x:1002:
admins:x:1003:mouli
[root@ip-172-31-95-150 ~]#
[root@ip-172-31-95-150 ~]# grep managers /etc/group
managers:x:1001:
[root@ip-172-31-95-150 ~]# usermod -aG managers mouli
[root@ip-172-31-95-150 ~]#
[root@ip-172-31-95-150 ~]# grep mouli /etc/group
managers:x:1001:mouli
mouli:x:1002:
admins:x:1003:mouli
[root@ip-172-31-95-150 ~]#
```

To add multiple users to a group at a time

```
[root@ip-172-31-95-150 ~]# gpasswd -M diksha,pallab admins
[root@ip-172-31-95-150 ~]# grep admins /etc/group
admins:x:1003:diksha,pallab
[root@ip-172-31-95-150 ~]#
```

To remove a user from a group

```
[root@ip-172-31-95-150 ~]# gpasswd -d diksha admins
Removing user diksha from group admins
[root@ip-172-31-95-150 ~]#
[root@ip-172-31-95-150 ~]# grep admins /etc/group
admins:x:1003:pallab
[root@ip-172-31-95-150 ~]#
```

Sudo user :

==========

Using sudo users we can allow a user to run administrative commands.this users can run all commands also we can allow for particular commands.

If the user can run all commands as root then the user is called “global Administrator” and if only few administrative commands are allowed then the user is known as “Custom Administrator”.

File Responsible for sudo : visudo

For creating user administrator : (useradd & passwd commands are allowed only)

visudo

```
%pallab ALL=(ALL)        /usr/sbin/useradd,/usr/bin/passwd
```

Save and exit

```
[pallab@ip-172-31-95-150 ~]$ sudo useradd kalu

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for pallab:
[pallab@ip-172-31-95-150 ~]$
[pallab@ip-172-31-95-150 ~]$ grep kalu /etc/passwd
kalu:x:1005:1007::/home/kalu:/bin/bash
[pallab@ip-172-31-95-150 ~]$
```

To create a global administrator

```
[root@ip-172-31-95-150 ~]# visudo

# %wheel        ALL=(ALL)       NOPASSWD: ALL
%mouli          ALL=(ALL)       NOPASSWD: ALL
```

Save and exit

```
[mouli@ip-172-31-95-150 ~]$ sudo useradd akash
[mouli@ip-172-31-95-150 ~]$ sudo passwd akash
Changing password for user akash.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
```

Special Permissions

==========

Sticky Bit :-

1. It is a special permission
2. It is applied on others section
3. It is used to restrict edit and delete of contents created by other users in a public location
4. Symbolic representation of sticky bit is “t”
5. Numeric representation of sticky bit is 1
6. It is by default applied on **/tmp**
7. Numerically it is situated before normal permissions.

Lab :

Applying sticky bit symbolically

```
[root@node1 ~]# chmod o+t /database/
```

Applying sticky bit numerically

```
[root@node1 ~]# chmod 1777 /database/
[root@node1 ~]#
```

Task :

1. create 3 users - user1 , user2 , user3 - password 123
2. Create a directory from root named /data
3. Now apply full permission
4. Now apply sticky bit
5. Test it from any 2 user

SGID :

1. Set group id
2. Applied on group owner section
3. Numeric value 2
4. Symbolic value “s”

5. It is done when we want group ownership on a parent location to be automatically inherited by the child objects.

```
create a directory named linux inside / . group owner will be admins. make sure
any user creates anything inside linux , group ownership must not be changed.
find all files created by admins group and save the output in a file
Admin_files
==========================================================================
1. mkdir /linux
2. groupadd admins
2. chown :admins /linux
3. chmod g+s /linux
4. chmod 777 /linux
5. create 1 file using rohit user
6. create one file using student user
7. find / -type f -perm SGID > admin_files
```

Sticky bit and sgid together

==========

Task :

```
[root@node1 ~]# # public folder : public1
[root@node1 ~]# # everyone must have full access
[root@node1 ~]# # no one can delete other's data
[root@node1 ~]# # group ownership for all files must be netadmins
```

Solution :

```
[root@node1 ~]# mkdir /public1
[root@node1 ~]# chmod 3777 /public1
[root@node1 ~]#
[root@node1 ~]# ls -ld /public1
drwxrwsrwt. 2 root root 6 Nov 30 00:34 /public1
[root@node1 ~]#
[root@node1 ~]# chown :netadmins /public1
```

SUID

==========

1. Set User ID
2. It is applied on owner section
3. Numeric representation is 4
4. It allows a normal user to run some administrative commands.
5. If we remove this permission , a normal user can not execute any administrative command.

```
[root@node2 ~]# whereis passwd
passwd: /usr/bin/passwd /etc/passwd /usr/share/man/man1/passwd.1ossl.gz /usr/share/man/man1/
[root@node2 ~]#
[root@node2 ~]#
[root@node2 ~]#
[root@node2 ~]# ls -l /usr/bin/passwd
-rwsr-xr-x. 1 root root 32648 Aug 10  2021 /usr/bin/passwd
[root@node2 ~]#
[root@node2 ~]# chmod u-s /usr/bin/passwd
[root@node2 ~]#
[root@node2 ~]# ls -l /usr/bin/passwd
-rwxr-xr-x. 1 root root 32648 Aug 10  2021 /usr/bin/passwd
[root@node2 ~]#
```

ACL : Access Control List

==========

1. It is used to assign permission to specific users and groups.
2. Permissions are for others remains same

Apply Read & Write Permission on only morium user

```
[root@node2 ~]# setfacl -m u:morium:rw /tmp/tuesday3/doc1
[root@node2 ~]#
```

Verification

```
[root@node2 ~]# getfacl /tmp/tuesday3/doc1
getfacl: Removing leading '/' from absolute path names
# file: tmp/tuesday3/doc1
# owner: student
# group: student
user::rw-
user:morium:rw-
group::r--
mask::rw-
other::r--
```

To remove all permission from jeet user

```
[root@node2 ~]# setfacl -m u:jeet:--- /tmp/tuesday3/doc1
```

To apply read write execute permission for a group named admins

Admins group members are - ishan and biraj

```
[root@node2 ~]# setfacl -m g:admins:rwx /tmp/tuesday3/doc1
[root@node2 ~]#
```

To remove acl from morium user

```
[root@node2 ~]# setfacl -x u:morium /tmp/tuesday3/doc1
[root@node2 ~]#
```

To remove acl from a group admins

```
[root@node2 ~]# setfacl -x g:admins /tmp/tuesday3/doc1
[root@node2 ~]#
```

Default ACL : Task :

==========

umask :

=========

1. When we create any file or directory in linux , these contents get automatic permissions.
2. umask assigns the default permission to the contents.
3. It is a 4 character value.
4. Default value is 0022
    a. 0 : denotes special permission
    b. 0 : permission for owner
    c. 2 : permission for group
    d. 2 : permission for others.

```
How umask is used to assign permissions
=========================================
for files :
============
full permission : 0  6  6  6
umask           : 0  0  2  2
-----------------------------------
default perm    : 0  6  4  4
-----------------------------------
0 : special permission
6 : read write (4+2) for owner
4 : read only for group owner
4 : read only for others
```

```
for directories
=================
Full Permission : 0  7  7  7
umask           : 0  0  2  2
=====================================
default Perm    : 0  7  5  5
========================================
0 : special permission
7 : read + write + execute = 4+2+1 = for owner
5 : read + execute = 4+1 = for group owner
5 : read + execute = 4+1 = for others
```

Changing umask value

==========

- Temporary change : valid till logout , when user will login again default value will be restored.
- Permanent Change : valid until changed

How to change umask value : Temporary

```
[student@node1 ~]$ umask 0222
[student@node1 ~]$
[student@node1 ~]$ mkdir test1
[student@node1 ~]$ ls -ld test1
dr-xr-xr-x. 2 student student 6 Dec  7 10:06 test1
[student@node1 ~]$
```

Permanent change of umask value for specific user

==========

1. Open .bashrc file : vim ~/.bashrc
2. Go to last line : press G
3. Press i to edit the file
4. Type umask<tab>000
5. Save and exit
6. Now check
7. Restart the system then also check

Creating a custom logout environment for a user

==========

1. vim ~/.bash_logout

2. Type the commands to be executed
3. Save and exit

Alias

======

```
[student@node1 ~]$ alias khoka='mkdir -p'
[student@node1 ~]$
[student@node1 ~]$ khoka rohit
[student@node1 ~]$ ls
a  database  exam  kk373  rohit  test1  weekend
[student@node1 ~]$
```

```
[student@node1 ~]$ alias ishan='ls -l'
[student@node1 ~]$
[student@node1 ~]$ ishan database/
total 8
-rw-r--r--. 1 student student 115 Nov 28 09:04 file1
-rw-r--r--. 1 student student  67 Nov 28 09:14 file2
drwxr-xr-x. 2 student student   6 Nov 28 08:48 mysql
[student@node1 ~]$
```

How to reset root password

==========

1. Restart the system
2. Interrupt the startup using up/down arrow key
3. Select rescue kernel
4. Press e
5. Go to the line starting with linux
6. Press ctrl+E to go to the last of the line
7. Type rd.break then ctrl+x
8. mount -o remount,rw /sysroot
9. chroot /sysroot
10. passwd root
11. Set password
12. Confirm password
13. Update timestamp : touch /.autorelabel
14. exit
15. exit